PORTFOLIO PROJECT · SYNTHETIC DATA

Supply Chain Analytics, end to end

Two operational apps, a scheduled ETL, a cloud data warehouse, and a Power BI story about distribution network health.

ERP (Supabase) · CRM (Aiven) · Python ETL · Neon warehouse · Power BI

Independent portfolio demo built on synthetic data, not affiliated with UniFirst.

THE 30-SECOND PITCH

I built the data before I built the dashboard: two apps create orders, then an incremental ETL feeds a cloud warehouse, and Power BI turns it into a five-page story that ends in a reorder list and a supplier to call.

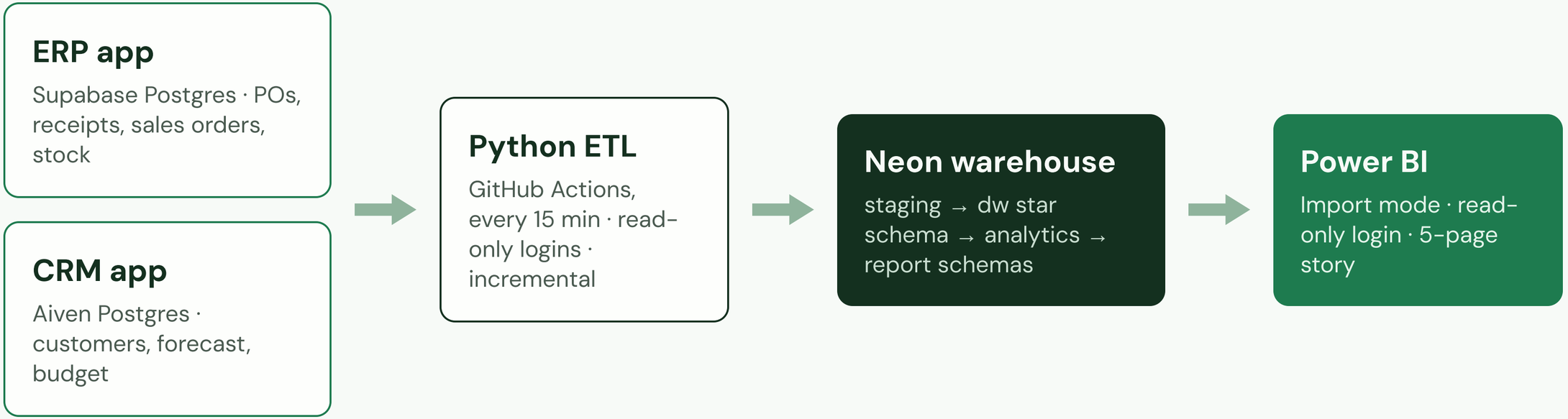
2 sources · **15-minute loads** · **2 report schemas** · **5-page story**

PART 1

How I built it

From the apps that create transactions to the warehouse that Power BI reads.

The architecture



ETL console on Vercel watches it all: load history, schedule, Run now, data-quality checks.

Every component runs in the cloud on a free tier · no secrets in code or GitHub

How I built it, in six steps

1

ERP app

Purchase, sales, receipts, stock on Supabase

2

CRM app

Customers, forecast, budget on Aiven

3

ETL

Incremental Python loads every 15 min

4

Warehouse

Star schema and views on Neon

5

Report schemas

KPIs tied to the analyst role

6

Power BI

DAX measures and a 5-page story

Each step was verified before the next: real queries against real tables, counts reconciled between source and warehouse.

Steps 1–2: the two source apps

SOURCE 1 · SUPABASE

ERP (SCM)

- Purchase orders and goods receipts
- Sales orders and inventory snapshots
- Products, suppliers, 3 distribution centers
- Writes only through checked functions

SOURCE 2 · AIVEN

CRM

- Customer accounts and pipeline
- Demand forecast by customer and SKU
- Sales budget with locked versions
- Links each ERP order to a customer and price

Why two systems? That's how real companies work, and it makes the warehouse join data that doesn't share a database.

Step 3: one ETL run, every 15 minutes

Extract
Read both sources with read-only logins

→

Compare
Row hash finds new and changed rows

→

Load
Upsert into staging, then the star schema

→

Check
Data-quality rules and a run log

2,038
rows loaded on the first run

0
rows on a rerun: loads are idempotent

393
updates picked up after CRM linked orders

Python · GitHub Actions schedule · advisory lock so two runs never overlap

Steps 4–5: the warehouse layers

staging raw copies of both sources

⬇

dw star schema: facts + dimensions

⬇

analytics calculation views and data quality

⬇

report schemas what Power BI reads

rpt_inventory

Distribution Network Health: stock status, fill rate, supplier scorecard, PO exceptions

rpt_demand

S&OP: demand forecast vs actual vs budget, forecast accuracy and bias

Logic lives in SQL views, so Power BI stays light and every number can be checked in the database

Built like production, on a small scale

Read-only

ETL and Power BI each have their own read-only login

No secrets

Passwords live in GitHub and Vercel secrets, never in code

Tested

Automated tests run on every change

Monitored

Console shows every run, its rows and data-quality results

A PROBLEM I SOLVED

The free GitHub scheduler skipped runs, so I added a heartbeat mode an outside timer can trigger. The console's schedule still decides whether a batch runs.

PART 2

The dashboard story

Distribution Network Health: five pages, one question each.

The story arc: one page per step

SITUATION

1 Network at a glance

Are we OK?

→

COMPLICATION

2 Inventory health

What is at risk?

→

IMPACT

3 Customer service

Are customers feeling it?

→

ROOT CAUSE

4 Suppliers

Why is it happening?

→

ACTION

5 Root cause and action

What do we do now?

Storytelling rules I used

- Every title states the finding, not the field name
- Red and amber only for problems; green for normal
- Every page ends with a question the next page answers

Situation → Complication → Impact → Root cause → Action

PAGE 1 · SITUATION · NETWORK AT A GLANCE

Customers are served well, but supply is shaky

$24.4K
inventory value

96.7%
order fill rate, below the 98% target

44.9%
of POs arrive on time

3
SKU-locations at risk

POINT TO

The headline banner, then the red cards. Close with: *“Fill rate looks fine at first glance. The next page shows why it won't stay that way.”*

Filtered to the three distribution centers (KY, OH, TN) · about 22 days of supply

PAGE 2 · COMPLICATION · INVENTORY HEALTH

Five SKU-locations need action, and the list is ready

2
stockouts

1
reorder now (under lead time + 7 days)

2
excess (over 90 days of supply)

78%
healthy (18 of 23)

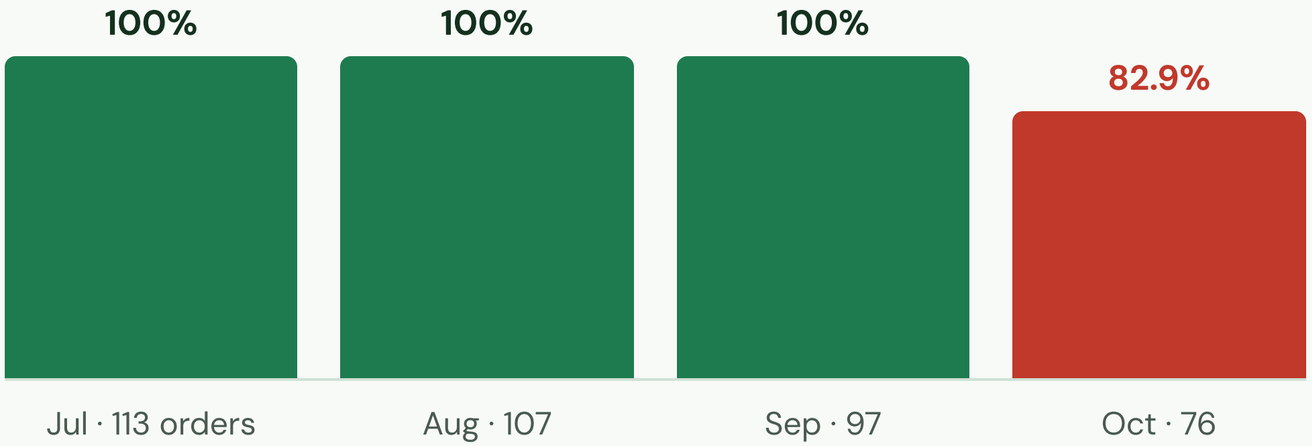
POINT TO

The action list, sorted by days of supply. *“This is the planner's to-do list: what to order, where, and how much.”*

Status rules: 28-day demand, inbound POs and each supplier's lead time

Fill rate held at 100% for three months, then fell in October

Order fill rate by month



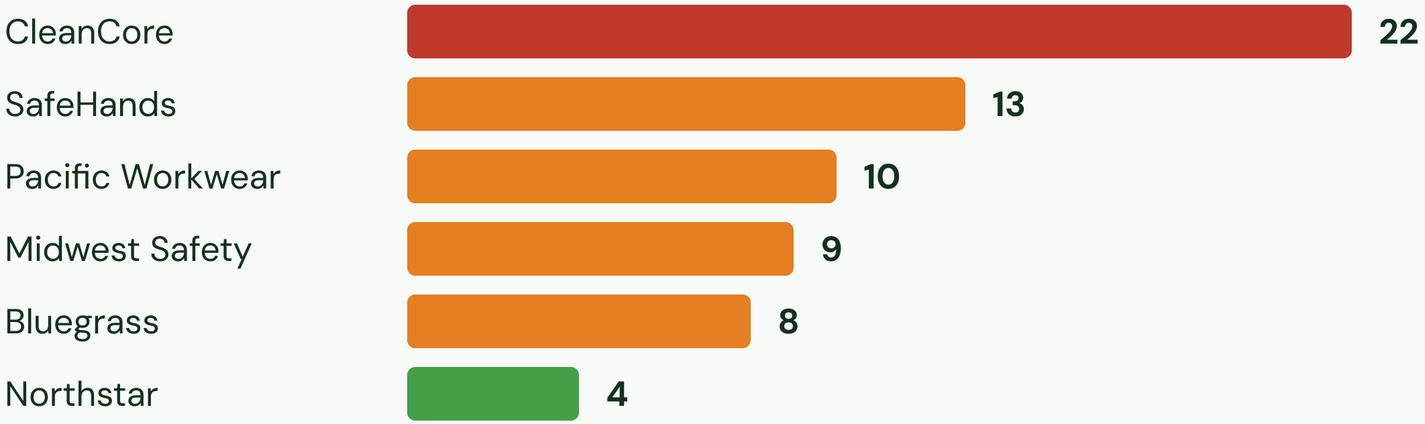
By distribution center (target 98%)

Kentucky	97.2%
Tennessee	96.8%
Ohio	96.2%

Bars start at zero; the October drop is 17 points · part of it is orders not yet shipped

Only 45% of POs arrive on time, and one supplier drives a third of exceptions

PO exceptions by supplier (66 in total)



120 POs

48 on time · **59** late · **7** overdue · **6** not yet due

On-time rate, worst and best

Pacific **23%** · CleanCore **40%** · Northstar **80%**

CleanCore: 22 of its 36 POs had an exception · Bluegrass POs run 11 days late on average

9 in 10 exceptions are late deliveries: $66K of $75K



Overdue 7 POs ($8,871) · Status mismatch 1 PO ($1,850, grey)

This week

Place the suggested reorders for the 3 at-risk SKU-locations (about $1.1K)

This month

Review CleanCore's late POs and agree recovery dates

This quarter

Raise safety stock for Pacific Workwear and CleanCore items

Exception PO value at estimated order cost · $75.2K across 66 POs (one PO is both overdue and a status mismatch)

Questions to be ready for

Why synthetic data?

It's a portfolio demo. The design works the same on real data; only the source logins change.

How is the ETL incremental?

Each row gets a hash; only new or changed rows load into the warehouse.

How do you know the numbers are right?

I checked every card against a SQL query on the warehouse, and the ETL runs data-quality checks.

Why Import mode?

Small data and fast visuals; the warehouse refreshes on a schedule anyway.

Why is $24.4K above the last snapshot?

The card sums the latest count per SKU-location, including spot counts after the full count.

What was hardest?

Relationships in Power BI: removing fact-to-fact links and fixing a date-table cycle.

Answer in one sentence, then offer to show it in the dashboard

Thank you

WHAT I'D BUILD NEXT

- Demand vs Actual vs Budget page from the warehouse's S&OP view
- Alerts when a SKU-location turns Stockout or Reorder now
- Supplier scorecard shared with buyers each month

ERP · CRM · ETL · Neon warehouse · Power BI — portfolio demo on synthetic data